Storms of High-energy Particles

João Tomás and Renato Viola, *Universidade Nova De Lisboa, Faculdade de Ciências e Tecnologia*

Abstract—In this paper we will explore how beneficial a CUDA implementation on a sequential program can be. Utilizing tools for code analysis, we aim to gain a comprehensive understanding of the specific areas where our program experiences slowdowns. Throughout this paper we will show charts of the first and final results, examples of changes that were made in the original code and our final view on this work.

Index Terms—Computer Science, Profiling, CUDA.

1 Introduction

In this paper, we will be tackling the parallelization of a sequential program simulating the effects of the bombardment of an exposed surface with waves of high-energy particles.

To our convenience, the provided program presents us this problem in a simplified manner by considering only a cross-section of the surface, represented by equally distributed control points on the outermost layer of the material. However, as expected from any problem involving particles, we can potentially be dealing with very large sets of data.

In this particular case, we must take into account not only the great number of particles that may be present, but also the number of control points, which could very well be in the tens or even hundreds of millions. In a realistic scenario, these operations can become quite resource-intensive and computationally demanding, hindering scalability and execution time in significant ways.

We aim to optimize this program and substantially improve its execution speed by harnessing the power of parallel processing in GPUs, whilst maintaining the very same functionality and correctness of the provided sequential code. To this end, we will be integrating CUDA in our approach.

We began by analysing the code to break it down for factorization, allowing us to subsequently employ profiling tools. This approach enabled us to pinpoint a good starting point for our CUDA optimizations.

November 12, 2023

2 Methodology

2.1 Code Factorization

After an initial analysis of the code, we had a decent understanding of its structure and functionality. Following this, we reorganized some of the code into well-defined functions, named after the section their respective code was in, in order to facilitate the profiling that would take place later on.

2.2 Profiling & CUDA implementation

With the code refactored and modularized, we can now make effective use of profilers to help us identify performance bottlenecks within the code. Instead of proceeding blindly and attempting to simply parallelize everything right away, we used *gprof* to help us decide where to prioritize our efforts - probably where the optimizations could yield the most significant improvements. For the parallelized version of the code, we utilized NVIDIA's own profiler for CUDA applications, *nvprof*, which provided us with a detailed breakdown of GPU kernel executions alongside other GPU-specific metrics.

In the following Pie charts we can see the percentages of time that each function was taking in the sequential version of the code with gprof, on tests 2, 7 and 8 with 30.000, 1.000.000 and 100.000.000 control points, respectively.

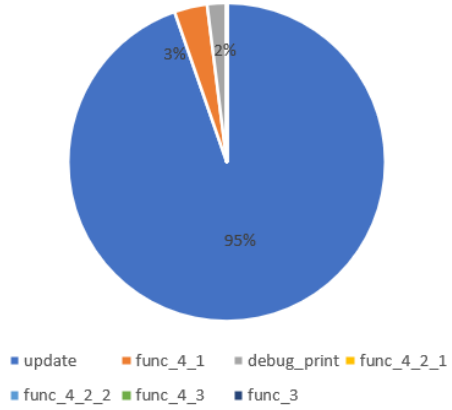


Fig. 1: Sequential Test 2 positions=30000

- *João Tomás and Renato Viola are with the Department of Computer Engineering, Universidade Nova de Lisboa, Faculdade de Ciencias e Tecnologia.*
  E-mail: jl.tomas@campus.fct.unl.pt E-mail: r.viola@campus.fct.unl.pt
- *J.Tomás and R.Viola are with FCT UNL.*

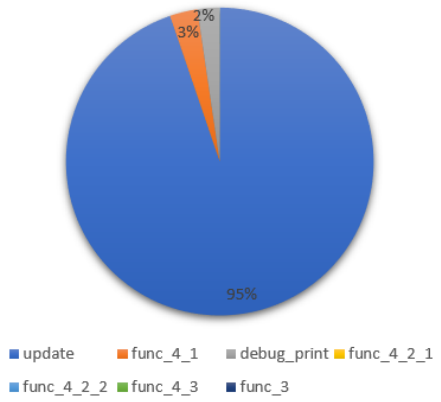


Fig. 2: Sequential Test 7 positions=1000000

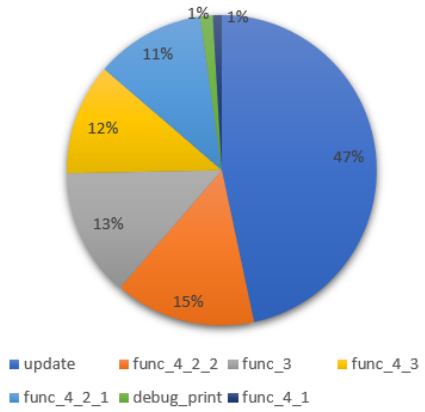


Fig. 3: Sequential Test 8 positions=100000000

From now on our focus moves to the parallelization of the more intense tasks that we were able to catch with the profiler. Following each optimization, we conduct tests and compare the results with the sequential version to assess the impact of the change on our program

3 Code Analysis & Examples

From the profiling on tests 2 and 7, we have detected that a large portion of the running time is due to the 'update' method being called many times. So this was our primary focus at this point.

As previously stated, `update` is being called a great number of times, so optimizing this function itself would be a step in the right direction. However, we weren't really sure how to optimize this method's logic without breaking it, so we left it as is. Our alternative was to optimize the function that calls `update` all these times, which is `func_4_1`.

Listing 1: Sequential code

```
void func_4_1(int i, Storm storms[], int layer_size, float *layer) {
    for (int j = 0; j < storms[i].size; j++) {
        float energy = (float)storms[i].posval[j * 2 + 1] * 1000;
        int position = storms[i].posval[j * 2];
        for (int k = 0; k < layer_size; k++) {
            update(layer, layer_size, k, position, energy);
        }
    }
}
```

We paralellized it by translating the function's most inner loop into a CUDA kernel:

- A thread is identified by its thread and block indices, as well as block dimension.
- We ensure that the thread is operating within the bounds of the layer with a conditional check.
- Each thread computes the `update` operation independently for a different cell.

Listing 2: CUDA code

```
__global__ void func_4_1(int layer_size, float *d_layer, int position, float energy) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;
    if (index < layer_size)
        update(d_layer, layer_size, index, position, energy);
}
```

After making this change, we noticed a very significant performance boost in the time our code took to run tests 2 and 7, with the control point values mentioned above.

We then moved on to the optimization of `func_4_2_2`, which was also shown to be somewhat of a hotspot.

Listing 3: Sequential code

```
void func_4_2_2(int layer_size, float *layer, float *layer_copy) {
    for (int k = 1; k < layer_size - 1; k++)
        layer[k] = (layer_copy[k - 1] + layer_copy[k] + layer_copy[k + 1]) / 3;
}
```

In this method, each element in the `layer` array is updated based on its own value and also the values of its adjacent neighbors in the `layer_copy` array, a stencil operation. Using shared memory here would have proved most useful, but at the time we hadn't researched the subject enough to implement it correctly. We approached this method very similarly to how we did for `func_4_1`, although with slight changes to the conditional check. Due to time constraints, we couldn't revisit this function and implement the shared memory approach.

Listing 4: CUDA code

```
__global__ void func_4_2_2(int layer_size, float *d_layer, float *d_layer_copy) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;
    if (index > 0 && index < layer_size - 1)
        d_layer[index] = (d_layer_copy[index - 1] + d_layer_copy[index] +
            d_layer_copy[index + 1]) / 3;
}
```

Aside from `func_4_3`, the remaining functions, `func_3` and `func_4_2_1`, were parallelized much in the same manner as `func_4_1`, varying only in the actual operation they execute.

The profiler of test 8 indicates that, `func_4_3` is the one taking the most time to execute. Unlike the other functions this one was not so direct, a naive implementation could lead to data races. Not only does the computation of each element depend on its neighboring elements, but also the innermost if-statement compares an element to the current known global maximum after asserting that it was a local maximum.

Listing 5: CUDA code

```
void func_4_3(int i, int layer_size, float *layer, float maximum[], int positions[])
{
    for (int k = 1; k < layer_size - 1; k++) {
        if (layer[k] > layer[k - 1] && layer[k] > layer[k + 1]) {
            if (layer[k] > maximum[i]) {
                maximum[i] = layer[k];
                positions[i] = k;
            }
        }
    }
}
```

The problem required some sort of synchronization, as we need to ensure all local maxima are being considered before we can effectively calculate the global maximum and its position. Our solution was to use a parallel reduction in order to find the local maximum for each block, and perform a final, much smaller reduction on the CPU to find the global maximum across all blocks. We chose Sequential Addressing over Interleaved Addressing as the strategy for our reduction, as it is bank conflict free and no more difficult to implement. To do this we got ideas from this [1] slides, the [2] github code and also from this youtube video [3].

- `sdata` and `sindices` are shared memory arrays which are used to store the values and indices of potential maxima found within the block;
- Each thread within the bounds of the layer loads an element from `d_layer_3` and its corresponding index into the shared memory structures;
- A loop iteratively checks for local maxima within the range of elements in shared memory, where each thread within the valid range of comparison compares elements separated by the stride distance and keeps the maximum value, as well as its index.
- The first thread in each block will write the block's maximum value and index in the global memory's arrays.
- Having every block's maximum, a final reduction is performed in the CPU to find the overall maximum and its position, which are then stored in `maximum[i]` and `positions[i]`.

Listing 6: CUDA code

```
__global__ void func_4_3_kernel(float *d_layer, float *d_maxValues, int *d_maxPositions,
    int layer_size) {

    __shared__ float sdata[THREADS_PER_BLOCK];
    __shared__ int sindices[THREADS_PER_BLOCK];

    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < layer_size) {
        sdata[tid] = d_layer[idx];
        sindices[tid] = idx;
    }
    __syncthreads();
    for (unsigned int s = THREADS_PER_BLOCK / 2; s > 0; s /= 2) {
        int strideIndex = tid + s;
        if (tid < s && strideIndex < layer_size - 1) {
            if (sdata[strideIndex] > sdata[strideIndex - 1] && sdata[strideIndex] >
                sdata[strideIndex + 1]) {
                if (sdata[tid] < sdata[strideIndex]) {
                    sdata[tid] = sdata[strideIndex];
                    sindices[tid] = sindices[strideIndex];
                }
            }
        }
        __syncthreads();
    }
    if (tid == 0) {
        d_maxValues[blockIdx.x] = sdata[0];
        d_maxPositions[blockIdx.x] = sindices[0];
    }
}
```

Listing 7: CUDA code

```
void func_4_3(int i, int layer_size, int nBlocks, float *d_layer, float *maximum, int
    *positions) {
    //MEMORY ALLOCATING

    func_4_3_kernel<<<nBlocks, THREADS_PER_BLOCK>>>
                (d_layer, d_maxValues, d_maxPositions, layer_size);

    //CUDA MEMCPY

    float max = -MAXFLOAT;
    int pos = -1;

    for (int j = 0; j < nBlocks; j++) {
        if (maxValues[j] > max) {
            max = maxValues[j];
            pos = maxPositions[j];
        }
    }

    maximum[i] = max;
    positions[i] = pos;
    //CUDA FREE...
}
```

4 Results

4.1 Final Profilers

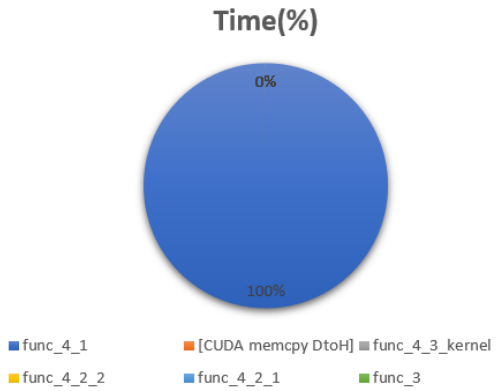


Fig. 4: CUDA Test 2 positions=30000

In this Pie chart we can see that the function where the program is taking the most time is the `func_4_1` with percentage close to 100%.

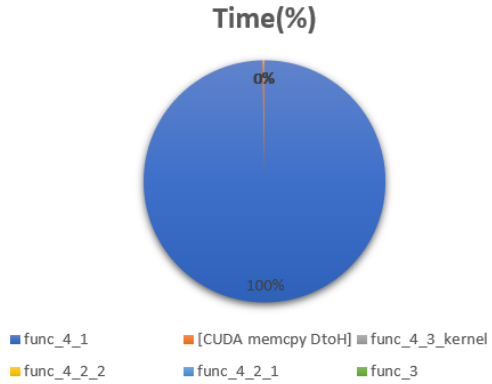


Fig. 5: CUDA Test 7 positions=1000000

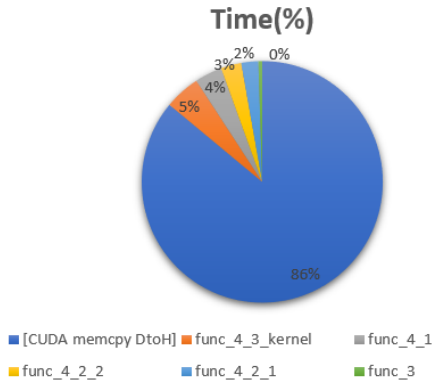


Fig. 6: CUDA Test 8 positions=100000000

In this last chart the percentages of each function get more spread, in this case the `func_4_1` is not the highest percentage and instead we get the `memcpy`, this shift can be attributed to the specific elements being tested under the conditions of test 8.

4.2 Test Machine

GPU:

```
Device 0: "NVIDIA GeForce GTX 1650 Ti"
  CUDA Driver Version / Runtime Version          12.2 / 11.5
  CUDA Capability Major/Minor version number:    7.5
  16 Multiprocessors (SM)
  Max Grid sizes: 2147483647, 65535, 65535
  Max Block sizes: 1024, 1024, 64
  Max Blocks per SM: 16
  Max threads per Block: 1024, Warp size: 32
  Total global memory: 4193984 MB
  Shared Mem per SM: 65536, Max ShMem per Block: 49152
```

CPU: AMD Ryzen 5 4600H
RAM: 8GB

4.3 Final Results

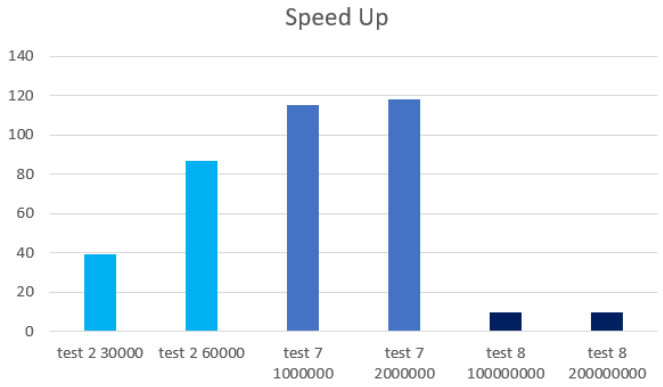


Fig. 7: Speed Up for Tests 2, 7 and 8

Here we have a chart that represents the final speed up of the different tests with two different inputs. The Y represents the speed up of the parallel version comparatively at the sequential one.

5 Group Working Methodology

During this project we used the "Visual Studio Code" text editor in which we occasionally coded simultaneously by using the extension "Live Share". Most of the time though, we split tasks between us and worked individually, while maintaining contact to stay informed about each other's progress. For example, while one of us was working on the script, the other would be handling the report or profiling the code. The report was mostly a joint effort of both of us. Overall, we believe we distributed the workload evenly and that our work complemented each other's, achieving a 50/50 split.

6 Acknowledgments

We would like to thank our colleagues Nelson Matos (60483) and Pedro Estróia (60691), who indicated us to Max Kotlan's Github repository where there are several implementations of maximum-value algorithms, using the parallel reduction pattern. The reduction pattern was the most challenging section of the project by far and it wasn't very easy to find references to anything other than sum reductions. It would have taken us a lot longer to implement this pattern,

had it not been for this reference, which was basically our guideline.

We'd also like to thank our professors for their availability to help.

7 Comments, criticisms, and suggestions

This was a very pleasant and interesting project to have worked on. Its theme itself was already captivating, and in terms of difficulty we found it quite well-balanced. It was challenging in some parts, while not being exhaustive. Not all optimizations could be approached the same way, some were varied and we were given the freedom to explore different ways to tackle them. As we had to push ourselves to properly learn and adapt concepts to our program, we grew more comfortable with parallel programming in general.

8 Conclusion

We started this project with the idea of getting a sequential program to have better performance by parallelizing it using a CUDA implementation. Throughout this paper we were able to show that our program did in fact run faster as shown in the results section. The recommended methodology that focused on the repeating steps of using a profiler to find the hot-spots/bottlenecks and then parallelizing, proved to be a highly effective strategy as we could shift our focus on was more important. Though we acknowledge that our solution has some room for improvement, we are very pleased with the results we have obtained and what we've learned as well.

References

[1] Mark Harris, NVIDIA Developer https://developer.download.nvidia.com/assets/cuda/files/reduction.pdf
[2] Max Kotlan, on GitHub https://github.com/MaxKotlan/Cuda-Find-Max-Using-Parallel-Reduction
[3] CoffeeBeforeArch, on Youtube https://youtu.be/bpbit8SPMxU